



Global OSINT Engine (Overwatch GEOINT) Project Report

1. Project Overview

The Global OSINT Engine (Overwatch) is a cloud-native, serverless Geospatial Intelligence (GEOINT) and Open Source Intelligence (OSINT) web application. The

platform provides real-time situational awareness by aggregating public mapping data and utilizing advanced Computer Vision (CV) to perform automated reconnaissance, anomaly detection, and tactical targeting on high-resolution satellite imagery. It is built to operate autonomously via an interactive web dashboard, presenting analysis in a futuristic, military-grade interface.

2. Technology Stack & Cloud Architecture

The application leverages a robust, serverless infrastructure on AWS to handle intense compute requirements and dynamic scaling without permanent server overhead.

Frontend

- **Web Technologies:** HTML5, CSS3, JavaScript (ES6)
- **Mapping Engine:** Leaflet.js (for rendering interactive base maps, plotting coordinates, and overlaying generated intelligence layers)
- **Hosting:** AWS S3 (Static Website Hosting)

Backend & Data Processing

- **Compute:** AWS Lambda (Serverless execution environment)
- **Core Languages:** Python 3
- **Computer Vision Engine:** OpenCV (cv2) and NumPy
- **Concurrency:** `concurrent.futures` (Multi-threading for parallel spatial tile fetching)

Persistence & APIs

- **Database:** AWS DynamoDB (NoSQL logging of scan events, timestamps, and detection metadata)
- **Ephemeral Storage:** AWS S3 (Storage and public serving of the OpenCV-processed tactical output images)
- **External APIs:**
 - **ArcGIS REST API:** Dynamic retrieval of high-resolution orbital satellite imagery
 - **OpenStreetMap (Overpass API / Nominatim):** Vector data extraction (MACRO OSINT) and geographic bounding box resolution

3. Core Functionalities & Operational Modes

A. The "Macro" OSINT Feed (Vector Reconnaissance)

The application connects to global OpenStreetMap databases to extract vector points of interest. To circumvent public API blocking and CORS limitations inherent in browser-based requests, the backend AWS Lambda acts as a **Stealth Proxy**. It filters geographic vectors into strategic categories: * Military Bases & Outposts * Power Plants & Energy Infrastructure * Airfields & Aviation Hubs * Transnational Highways

B. The "Micro" Scan (Computer Vision & Tactical Analysis)

When a target is selected, the application shifts into Micro Scan mode, triggering the Python OpenCV pipeline in AWS Lambda. The system supports distinct, specialized computer vision algorithms based on the target type:

1. Maritime Mode (Suez Canal / Alexandria Port)

- **Dynamic Sub-Tiling:** For massive continental targets like the Suez Canal (160km), a single API request drops resolution heavily. The system mathematically divides the geographic bounding box into multiple sectors, uses parallel threads to fetch them simultaneously in high resolution, and mathematically stitches them back together (`cv2.vconcat`).
- **Advanced Water Segmentation:** Utilizes an **Adaptive Otsu's Threshold** to dynamically separate land from water regardless of lighting variations or exposure (handling both the dark waters of the Suez and the bright coastal waters of Alexandria).
- **Topological Noise Reduction:** Applies a bitwise masking technique to dim all non-water pixels by 70%, forcing visual focus strictly onto the maritime activity.
- **Artifact Rejection Filtering:** Analyzes contours using extreme structural constraints. It evaluates bounded boxes (`minAreaRect`), aspect ratios (`> 1.8` , `< 8.0`), structural extent (`> 0.45`), and physical width to isolate true vessel signatures while mathematically rejecting islands, shorelines, docks, and map-stitching grid artifacts.
- **True Shoreline Wrapping:** Uses Canny Edge Detection and morphological dilation to trace the exact geographic shoreline with a cyan contour, intentionally erasing image borders to prevent artificial boundary lines from cutting across the open water.

2. Aviation Mode (Cairo International Airport)

- **Smart Imagery Fallback:** The frontend Leaflet map implements a custom `L.TileLayer.SmartEsri` override. It dynamically detects when the primary ArcGIS satellite server returns empty gray "no-data" placeholders (by analyzing the byte size of the fetch response in real-time) and seamlessly substitutes OpenStreetMap fallback tiles. This prevents the user from hitting a "gray wall" when zooming deeply into remote areas.
- **Adaptive Morphology:** The backend utilizes Top-Hat and Black-Hat morphological transforms (via `cv2.morphologyEx`) to isolate small, bright, or dark metallic structures (aircraft fuselages) against complex concrete runway backgrounds.
- **Convexity Defect Analysis:** To distinguish aircraft from square airport buildings, the system calculates the convex hull of each detection and measures structural defects (the spaces between the wings, tail, and nose) to mathematically prove the cross-like shape of an airplane.

4. UI/UX Design

The interface is designed with a dark, cyberpunk/military aesthetic (`#000000` backgrounds with cyan and green terminal accents). The interactive map uses a dynamically updating right-side intelligence log ("Live Intel Feed") to display scanning progress, detection counts, and API cache status, completing the immersive GEOINT dashboard experience.

5. System Architecture Diagram

flowchart TD

```
User((User / Web Browser)) -->|Interacts with UI| Frontend[AWS S3: Static Le  
Frontend -->|POST Request (Coords/Filter)| APIGW[AWS API Gateway]  
APIGW -->|Triggers| Lambda[AWS Lambda: Python CV Engine]
```

```
subgraph Serverless Cloud Backend
```

```
    Lambda
```

```
    DynamoDB[(AWS DynamoDB: Intel Logs)]
```

```
    S3_Images[(AWS S3: Processed Imagery)]
```

```
end
```

```

subgraph External Data Providers
    ArcGIS[ArcGIS REST API: Satellite Tiles]
    OSM[OpenStreetMap: Vector Data]
end

Lambda <-->|Multi-threaded Fetch| ArcGIS
Lambda <-->|OSINT Stealth Proxy| OSM
Lambda -->|Mathematical Stitching & OpenCV| Lambda
Lambda -->|Uploads Tactical Image| S3_Images
Lambda -->|Logs Target & Anomalies| DynamoDB
Lambda -->|Returns JSON & Image URL| Frontend

```

6. Executive Summary & Architecture Overview

Project Abstract: The Global OSINT Engine is a fully serverless Geospatial Intelligence (GEOINT) and Computer Vision platform powered by AWS and Python.

The Challenge: Running complex Computer Vision algorithms (such as Adaptive Otsu Thresholding and Canny Edge Detection) on massive, high-resolution satellite arrays traditionally requires expensive, dedicated GPU instances. The objective was to engineer a real-time anomaly detection engine that operates entirely serverless, executing on the fly without persistent compute costs.

The Architecture & Solution: A 100% cloud-native architecture was designed to handle the scale and processing requirements:

- * **Frontend:** A tactical Leaflet.js dashboard hosted statically on AWS S3. It features a custom "Smart Imagery Fallback" that dynamically swaps missing satellite data with OpenStreetMap tiles on the fly based on network response byte-size analysis.
- * **Backend Compute:** AWS Lambda handles the primary processing load. When a target is selected, Lambda utilizes concurrent threading (`concurrent.futures`) to fetch massive orbital grids from ArcGIS, stitches them together in-memory, and executes a custom OpenCV pipeline to mathematically detect structural anomalies, such as aircraft and maritime vessels.
- * **Data Persistence:** The processed tactical feeds are instantly pushed to a backend AWS S3 bucket, while the detection metadata and intelligence logs are recorded in AWS DynamoDB for long-term auditing.

The Result: A highly scalable, on-demand GEOINT dashboard that dynamically tracks global chokepoints and infrastructure without relying on a single permanent server.

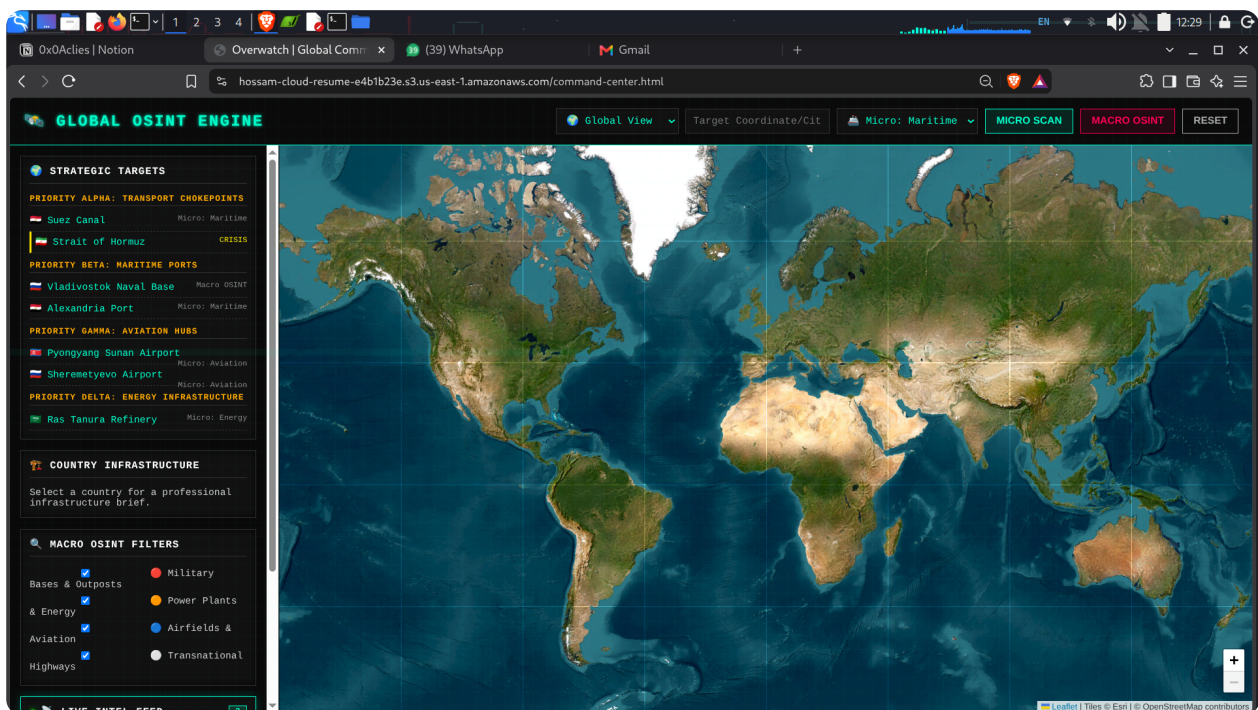
Tech Stack: Python 3, OpenCV, AWS (Lambda, S3, API Gateway, DynamoDB), JavaScript (Leaflet).

7. Application Showcase (Gallery)

Here are the latest operational screenshots of the Global OSINT Engine in action:

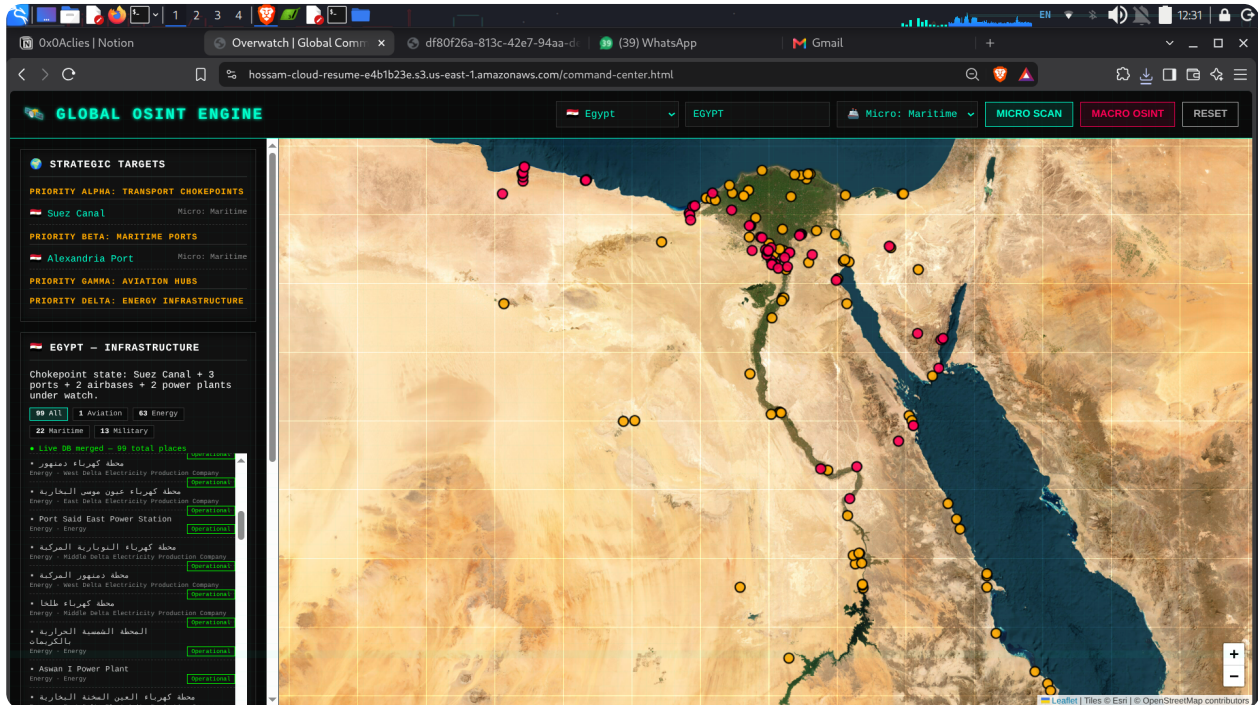
Global Strategic View

The default global command view, plotting worldwide chokepoints, naval bases, and energy infrastructure.



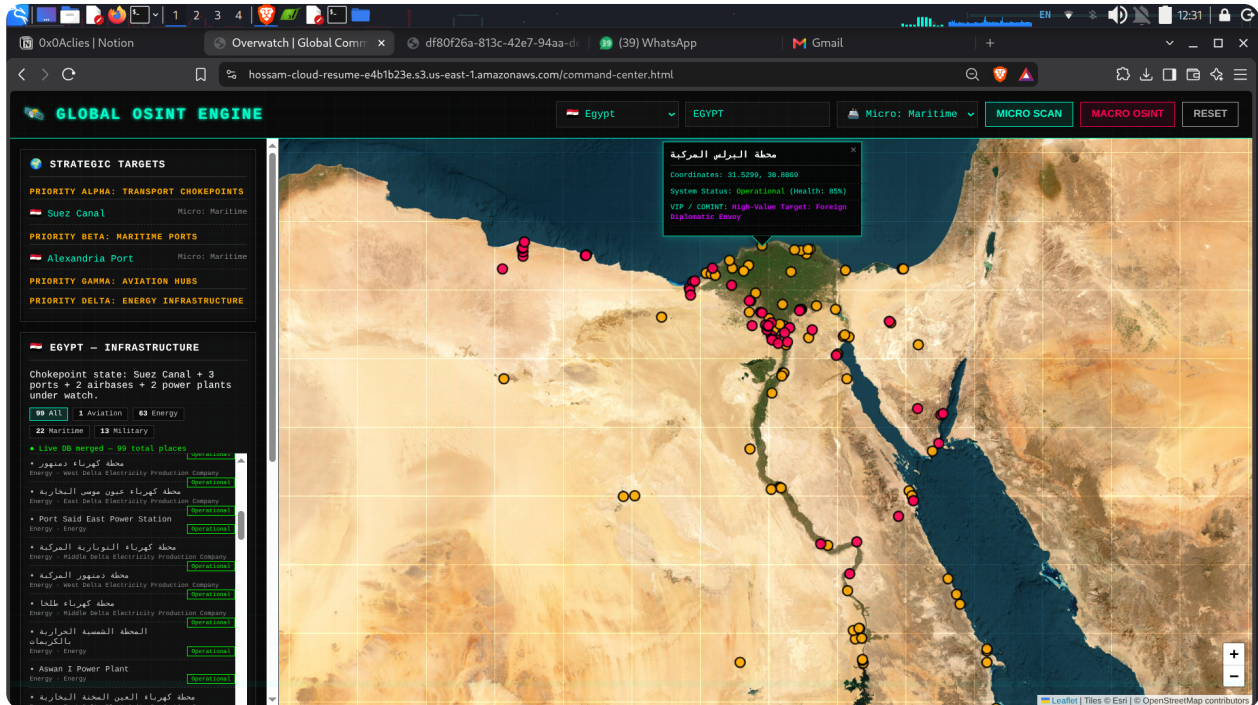
Macro OSINT Feed (Infrastructure Mapping)

The system pulling vector data from the stealth proxy, successfully mapping 99 discrete infrastructure targets across Egypt (Power Plants, Airbases, Military Sites).



Interactive Target Analysis

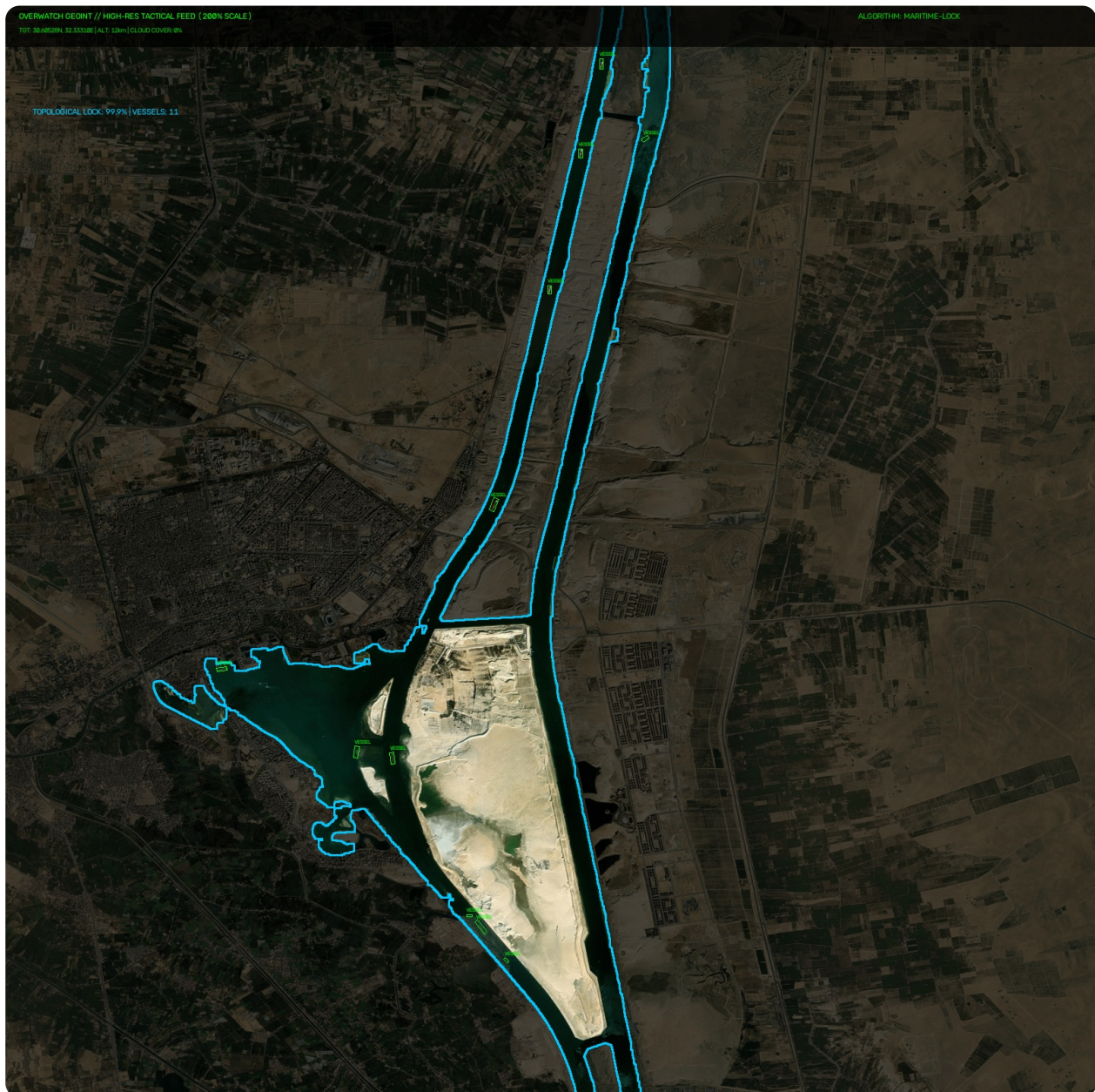
Detailed intelligence tooltip revealing coordinates, operational health, and VIP COMINT status for specific energy facilities.



Maritime Scan: Suez Canal

The Computer Vision engine executing the maritime pipeline. Adaptive Otsu Thresholding and True Shoreline Wrapping successfully isolate the waterway while

mathematically rejecting surrounding terrain and topographic artifacts.



Aviation Scan: Cairo International Airport

The Computer Vision engine executing the aviation pipeline. Top-Hat morphological filters and Convexity Defect Analysis accurately detect and track independent aircraft

fuselages across complex concrete runway environments.

